

Computer Hardware ■ Lecture 8: Practice with Threads

Task 5: The pizzeria

Three cooks make three kinds of pizza, and sometimes they need the same ingredients. The owner says that something is wrong with the kitchen software: some evenings the kitchen stops, and on other evenings the number of pizzas on the board does not match the till. This program has **two** bugs. Find both, tell them apart and fix them.

This task joins Tasks 1 to 4: one race condition and one deadlock in one program.

The kitchen

Every ingredient is a mutex: only one cook at a time takes from it. A cook takes the ingredients in their own order, holds all of them until the pizza is baked, and then gives them back.



After every pizza the cook adds 1 to the number on the board, pizzas. Every cook makes 10 pizzas, so at the end the board should show 30.

Ready code (in kitchen.h, we do not look inside)

Functions

take(m)	take an ingredient: lock its mutex
bake()	bake the pizza: 2 ms
write_on_board()	write the number on the board: 2 ms
serve(ms)	bring the pizza to the table: A 20 ms, B 30 ms, C 40 ms

Constants

PIZZAS	10: every cook makes 10 pizzas
--------	--------------------------------

The code

The template is in [Task5](#). This is pizzeria.cpp:

```
// every ingredient is a mutex: one cook at a time takes from it
std::mutex tomato, cheese_sauce, mozzarella, cheddar, jalapeno, pepperoni;

int pizzas = 0;           // the number on the board: all pizzas so far

void add_to_board() {     // pizzas++, written out
    int n = pizzas;       // read the number on the board
    write_on_board();     // (takes a moment)
    pizzas = n + 1;       // write the new number
}

void cook_a() {           // Fat Peppa
    for (int i = 0; i < PIZZAS; i++) {
        take(tomato); take(mozzarella); take(pepperoni);
        bake();
        pepperoni.unlock(); mozzarella.unlock(); tomato.unlock();
        add_to_board();
        serve(20);
    }
}

void cook_b() {           // Fat Cheese
    for (int i = 0; i < PIZZAS; i++) {
        take(cheese_sauce); take(mozzarella); take(jalapeno); take(cheddar);
        bake();
        cheddar.unlock(); jalapeno.unlock();
        mozzarella.unlock(); cheese_sauce.unlock();
        add_to_board();
        serve(30);
    }
}

void cook_c() {           // Diavola
    for (int i = 0; i < PIZZAS; i++) {
        take(jalapeno); take(tomato); take(mozzarella); take(pepperoni);
        bake();
        pepperoni.unlock(); mozzarella.unlock(); tomato.unlock(); jalapeno.unlock();
        add_to_board();
        serve(40);
    }
}

int main() {
    std::thread a(cook_a), b(cook_b), c(cook_c);
    a.join(); b.join(); c.join();
    std::cout << "pizzas on the board: " << pizzas
                << " (should be " << 3 * PIZZAS << ")\n";
}
```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Run the program 10 times. Write down what happens each time: it hangs, it prints a wrong number, or it prints 30. (A program that hangs: stop it with Ctrl+C.)
- 2 Find the race condition on pizzas. Which lines, and what exactly goes wrong?
- 3 Fix the counter. `std::atomic` or a mutex? Why?
- 4 Draw who waits for whom when the program hangs, and find the circle.
- 5 Only cook B uses the cheese sauce and the cheddar. Are they part of the deadlock?
- 6 Cooks A and C both use pepperoni. Is it part of the deadlock?
- 7 Choose one order for everybody. A pizza is made from the bottom up: sauces first, then cheeses, then toppings: tomato sauce, cheese sauce, mozzarella, cheddar, jalapeño, pepperoni. Change the order of cooks B and C to follow it.
- 8 Run the program 10 times again. Does it ever hang? Is the number always right?